# Project 2: Smarter BlinkIt

Most shopping apps today are just digital catalogs. If you want to "cook an Italian dinner," you have to search for every single item—pasta, sauce, cheese—one by one. It's slow and boring. On the other side, small shop owners have to manually type in every single product they sell, which leads to mistakes and "out of stock" headaches. We need a "Smart Marketplace"—a website that understands what you want to do, not just what you type.

## Goal:

Your goal is to **build a web application** that connects buyers and local sellers. Instead of just a search bar, you'll build an **"AI Shopping Assistant"** that fills your cart for you and a "**Barcode based Inventory Modification System".**

## The 4-Stage Approach:

### Stage 1: The Foundation

**Goal:** Build a working website where users can find what they need.

- **Dual-Login System:**
  - Create a secure login where the app knows if you are a "Buyer" or a "Seller." Each should see a different dashboard.
  - Face recognition for login and authorization purposes. Look up *Open-CV.*
- **"Intent" Search:**
  - Instead of just matching words, try to make the search smart. If a user types "I have a cold," the website should suggest **Honey** or **Ginger Tea** automatically. *Research Tip: Look up "Semantic Search" to see how this works!*
- **Local First:**
  - The website should detect the user's location and auto place the order to the closest shops first so that delivery is fast and cheap.
- **Barcode based Inventory Modification:**
  - Sellers are busy. Build a feature where a seller can share barcodes on boxes to update their inventory with ease.
- **Dummy Payment:**
  - Buyers can complete orders using a test payment flow to simulate a secure real-world checkout experience. (Tip : "Razorpay" provides test payment integration)

### Stage 2: The Automator

**Goal:** Let the AI do the heavy lifting.

- **The "Recipe" Agent:**

    - Imagine typing "Make Pizza for 4 people." The website should automatically find the flour, cheese, and tomato sauce of the required amount from nearby shops and put them all in your cart in one click.
- **Similar Items Suggestion:**
    - When a buyer views a product, the app should automatically show helpful suggestions. For example, if someone is viewing a pasta brand that is out of stock, the system can suggest other pasta brands. It can also show items people usually buy together, like pasta sauce or cheese.
    - ***Important Usage*** *: Use Databases that store data as Graphs (**Neo4j**) Store products as nodes and connect them with simple relationships like* `SIMILAR_TO` *and* `BOUGHT_WITH`*. Use basic Cypher queries to fetch the most connected products.*

## Stage 3: The Orchestrator (Logistics & Quality)

**Goal:** Make sure the order is right and the route is fast.

- **Smart Cart Splitting:**
    - If Shop A has the pasta but Shop B has the cheese, the website should be smart enough to split the order and find the best way to deliver both to the user's house.
- **Live Storeboard:**
    - Create a real-time dashboard for the owner that shows which items are selling the fastest and which shops are the "Top Rated" in the city right now.

## Bonus Stage: God Mode

**Goal:** Prediction based features

- **The "Money Map":** A visual chart that shows which neighborhoods are buying the most and where new shops should open and also which types of items are majorly sold in the neighbourhood.
- **Smart Product Pairing:** Integrate a prebuilt model which predicts which two products can be displayed together based on whether they are majorly sold together in the past. For more info search for *Beer and Diaper* case study to understand how such analyses are helpful and how to implement it using the prebuilt ML model.
  *Checkout: [Hugging Face](#) to find a suitable model.*

---